

Committee sizing under the (quorum-1)x2 cap: the tables, the decision the cap forces, and 3A/3B by example

Reply to your 2026-07-03 note. Contents: (1) your cap is exactly the right invariant, and enforcing it forces one concrete design decision that I spell out; (2) the representativeness table; (3) the stall tables at 20/35/45% sleepy, with the true cost of a stall measured from the code; (4) the table you asked for implicitly, malicious capture, which turns out to be the binding constraint on committee size; (5) dynamic vs fixed and the value I suggest; (6) 3A explained again with a worked example and a two-tier fix; (7) 3B the same way, and one mechanism that closes both. Every number is exact hypergeometric or exact binomial from `committee-sizing-tables.py` (checked in next to this note, pure Python, no dependencies); nothing is simulated or estimated.

1. Your cap is the right invariant, and it forces one design decision

The invariant. If two blocks at the same height each collect a quorum Q from an eligible set of size K , the two signer sets overlap in at least $2Q - K$ members. Your cap $K \leq (Q-1) \times 2$ makes that overlap at least 2: no two conflicting certificates can form unless at least two members sign both. Honest members with a durable signing record never sign twice, so under the cap an honest-node double-certification is impossible by arithmetic, and even one Byzantine equivocator is tolerated.

Why the code violates it today. Membership is a per-staker threshold on a private VRF output: `PosVrfSlot(beta, weight, total) < 100` (pos.cpp:364). Each staker's beta is independent, so the eligible count is a random variable with mean 100, while the quorum stays pinned at 51 (validation.cpp:2254). Once the pool exceeds 100 stakers, the eligible set exceeds 102 on roughly 40% of heights and two disjoint 51-quorums fit inside it.

The decision the cap forces. The natural fix, "take the 100 best-ranked stakers," cannot be bolted onto the current sortition, and the code itself says why (pos.h:323): "with private sortition nobody can rank stakers (each beta is secret until published), so membership is threshold-based." A sleepy member never publishes its beta, so no node can ever know the true top-100, and any cap computed from the betas a node happens to have seen is node-local, which is exactly the disease we are curing. There are only two honest ways out, and the numbers decide between them:

Option A, public committee. Compute membership from public data: the committee for height H is the first K entries of the deterministic schedule `PosSchedule(registry, seed)`, the ranking by $H(\text{seed} \parallel \text{pubkey}) / \text{weight}$ that the code already uses for leader ordering (pos.cpp:87-118, PosCommittee at pos.cpp:130). The seed is already `H(parent_anchor_hash, height)` (PosSeedForChild, pos.cpp:292-304): it comes from Bitcoin's proof of work, so a Sequentia producer cannot grind it, and it is fixed per height on every node. Every node then agrees on exactly who the K members are, $K = \min(\text{pool}, \text{cap})$, and the eligible set can never exceed the cap. This is your formula, enforced by construction.

Option B, keep private sortition and out-margin the variance. Keep the threshold rule and raise the quorum until two disjoint quorums are statistically impossible even at the top of the size distribution. Exact numbers, requiring $P(\text{eligible} \geq 2Q)$ below $1e-9$ per height and then asking what happens at 35% sleepy:

expected committee	safe quorum	quorum as % of committee	P(stall) at 35% sleepy
100	83	83%	98.2% (chain dead)
500	320	64%	38.3% (chain dead)
1000	597	60%	1.64% (stall every 30 min)
2000	1135	57%	0.0001% (stall every ~11 months)
3000	1663	55%	negligible

Private sortition with real safety margins needs a committee of two to three thousand. That is Algorand's regime, and it is why Algorand runs committees of that size. It contradicts everything you want: small, efficient, launchable with few nodes. So Option A is the answer, and the private VRF stays where it still earns its keep, leader election (the leader's slot remains secret until the proposal appears, which keeps leader-targeted DoS blind).

What Option A costs. The committee for height $H+1$ becomes computable as soon as block H exists, and, since the seed only changes when the parent's Bitcoin anchor changes, an observer can project committees a few minutes ahead within one Bitcoin block interval. That is a targeted-DoS surface that private sortition does not have. Two mitigations: members are pubkeys, not IP addresses, and mapping key to machine is the hard part for an attacker; and the precedent is mainstream, Ethereum publishes its attestation committees an epoch in advance. I judge the trade clearly worth it; you should confirm you do too.

What Option A gives back, beyond the safety fix. Two things I did not expect when I started measuring:

- 1. The certificate shrinks about 100x.** Today each certificate carries ~257 bytes per member (chainparams.cpp:444, sized at $300 \times 100 + 2000 = \sim 32$ KB per block): pubkey, VRF proof, BLS pubkey, proof of possession. All of it exists only to prove private-sortition membership. With a public committee the member set is known to every validator from the registry and the seed, so the certificate reduces to the leader signature, one 96-byte BLS aggregate, and a K-bit signer bitfield (~13 bytes at $K=100$), with BLS keys registered once at staking time instead of re-proven in every block. Cost per block drops from ~32 KB to ~300 bytes. This removes the main efficiency penalty of larger committees, which matters for section 5.
- 2. The stall probability drops 23x to 59x at the same nominal size.** Under the threshold rule the committee size itself varies, and both variances count against the fixed quorum. Exact comparison at 35% sleepy, target 100, quorum 51:

staker pool	threshold sortition (today)	fixed-size cap (Option A)	improvement
500	2.39% (stall every ~20 min)	0.041% (every ~20 h)	59x
1000	2.80%	0.083% (every ~10 h)	34x
10000	3.17%	0.14% (every ~6 h)	23x

So the cap is not only the safety fix, it rescues liveness at scale: with today's rule and a large pool, a 35%-sleepy network would stall every twenty minutes.

Spec details that come with it: quorum must derive from the actual K, not the nominal cap (today validation pins quorum to PosQuorum(100) even if the pool is 60, which would demand 51 of 60, an 85% quorum; with $K = \min(\text{pool}, \text{cap})$ and $Q = \text{PosQuorum}(K)$, a 60-staker launch runs 31-of-60). For odd K a bare majority gives overlap 1 instead of 2; if you want the one-equivocator margin at every size, use $Q = \text{floor}((K+1)/2) + 1$, which is identical to the current rule at every even K including 51-of-100. All of this is consensus-level: it cannot be switched on under the live chain (existing blocks would re-derive membership and fail) and folds into the planned re-genesis. One deployment note that makes it painless: with any cap of 100 or more, a re-genesis chain over the current 100-staker fleet behaves identically to today in normal operation, $K = \min(100, \text{cap}) = 100$ and $Q = 51$, so the fix changes nothing observable until the pool actually grows.

2. Representativeness: how far the committee drifts from the staker distribution

The quantity that answers your EU/US/Asia question is the standard deviation of a group's share of the committee under sampling without replacement (hypergeometric), with the finite population correction. I report the SD directly, in percentage points, for a group that is one third of the population; variance is its square and less readable. The worst case single region, one holding 50% of stakers, multiplies these numbers by 1.061; smaller regions come in below them.

Table 1. SD of a 33.3% region's committee share, percentage points. Rows committee size, columns staker population. n/a where the committee cannot exceed the population.

committee	100	500	1000	10000	50000
20	9.48	10.34	10.44	10.53	10.54
30	7.24	8.35	8.48	8.59	8.60
50	4.74	6.33	6.50	6.65	6.66
80	2.37	4.84	5.06	5.25	5.27
100	0.00	4.22	4.47	4.69	4.71
250	n/a	2.11	2.58	2.94	2.97

committee	100	500	1000	10000	50000
500	n/a	0.00	1.49	2.05	2.10

The same numbers as 95% ranges, population 10,000: on 95% of heights a region that is exactly one third of the stakers holds this share of the committee:

committee	95% range of the region's share
30	16.5% to 50.2%
50	20.3% to 46.4%
100	24.1% to 42.5%
150	25.8% to 40.8%
200	26.9% to 39.8%
250	27.6% to 39.1%
500	29.3% to 37.4%

Readings:

- Your intuition that "a sample of 100 in general is good" is exactly the classical result, and the table confirms why: away from the left column the numbers barely move as the population grows. Once the population is more than about ten times the committee, accuracy is set by committee size alone. This is the fact that settles dynamic-vs-fixed in section 5.
- The population only matters when the committee is a large slice of it (left column): a 100-of-100 committee is the population, error zero. This is the launch regime, and it is the best one.
- The knee is around 100 to 250. Committee 100 holds a one-third region within roughly 24% to 42%; committee 250 within 28% to 39%; going to 500 buys little more.
- One caveat: sortition is stake-weighted, so the committee mirrors the distribution of stake, not of headcount. If Europe has a third of the stakers but half the stake, expect half the committee. The tables are per unit of stake (equivalently, equal-weight stakers).
- Representativeness is not only fairness: sleep is correlated with geography (a region's night, a cloud provider's outage). A committee spread like the population always has members awake somewhere, which is what makes the independence assumption in the next section approximately true.

3. Sleepy nodes: the stall tables, and what a stall actually costs

Model, per your hypergeometric framing: the population has N stakers, a fraction s of them sleepy; the committee of size n is drawn without replacement; sleepy members in committee follow

Hypergeometric(N , $\text{round}(sN)$, n). The height stalls when awake members cannot reach quorum, i.e. when at least $n - Q + 1 = \text{ceil}(n/2)$ members are asleep.

Before the tables, the cost of a stall, measured from the code, because it is worse than "try again next block." The election seed for a height is fixed (parent anchor hash plus height, `pos.cpp:292`), so a stalled height keeps the same committee; rounds rotate the leader (`pos_producer.cpp DriveRound`) but never the committee. The stall therefore persists until the escaping-stall valve opens: the chain may accept a below-quorum certificate, as low as a single member, only once the new block's Bitcoin anchor is at least 3 blocks past the parent's (whitepaper section 3.8; `POS_ESCAPING_STALL_ANCHOR_GAP`, `pos.h:338`; `validation.cpp:2257`). Three Bitcoin blocks average about 30 minutes. So each stall is a roughly half-hour outage, after which the chain limps forward on sub-quorum blocks; those escape blocks are deliberately second-class, they are not immediately final (finality needs a full quorum, `validation.cpp:3168`) and they lose fork-choice to any quorum-certified sibling (the countersignature key, `validation.cpp:136`). Full-stall throughput floor: about one block per 30 minutes. Graceful, but you do not want to live there.

Table 2. P(height stalls), 20% sleepy.

committee	100	500	1000	10000	50000
20	0.065%	0.21%	0.23%	0.26%	0.26%
30	0.0004%	0.014%	0.018%	0.023%	0.023%
50	0	4e-7	0.0001%	0.0002%	0.0002%
80	0	2e-11	3e-10	2e-9	2e-9
100	0	1e-14	9e-13	2e-11	2e-11
250	n/a	0	6e-38	6e-27	3e-26
500	n/a	0	0	9e-54	4e-51

Table 3. P(height stalls), 35% sleepy.

committee	100	500	1000	10000	50000
20	9.64%	11.7%	11.9%	12.2%	12.2%
30	3.47%	5.93%	6.23%	6.49%	6.51%
50	0.15%	1.56%	1.81%	2.04%	2.06%
80	0	0.19%	0.29%	0.40%	0.41%
100	0	0.041%	0.083%	0.14%	0.14%
250	n/a	1e-12	1e-8	6e-7	8e-7

committee	100	500	1000	10000	50000
500	n/a	0	8e-24	1e-12	3e-12

Table 4. P(height stalls), 45% sleepy.

committee	100	500	1000	10000	50000
20	39.9%	40.7%	40.8%	40.9%	40.9%
30	33.0%	35.1%	35.3%	35.5%	35.5%
50	21.1%	27.4%	27.9%	28.3%	28.4%
80	3.75%	19.5%	20.6%	21.5%	21.5%
100	0	15.6%	17.0%	18.2%	18.2%
250	n/a	1.54%	3.92%	6.14%	6.33%
500	n/a	0	0.091%	1.21%	1.36%

Translated into expected time between half-hour outages (population 10,000, 30-second blocks, 2,880 heights/day):

committee	20% sleepy	30%	35%	40%	45%
50	every ~6 months	hours	continual	continual	continual
100	never (60,000 yr)	every ~17 days	every ~6 h	continual	continual
150	never	every ~5 yr	every ~3.4 days	hourly	continual
200	never	every ~540 yr	every ~45 days	every ~4 h	continual
250	never	never (60,000 yr)	every ~1.6 yr	every ~11 h	continual

Readings:

- At 20% sleepy everything from committee 50 upward is effectively immune.
- 35% sleepy is where sizes separate: committee 100 takes a half-hour outage every six hours, roughly 10% downtime; committee 200 one outage per six weeks; committee 250 one per year and a half.
- At 45% sleepy no practical committee survives: the transition these curves cross sits at 50% asleep, and the closer s gets to it the more the sample straddles it. The chain then runs at the escape-valve floor regardless of size (thousands of members would be needed to change that).

Treat 45% as a disaster mode the protocol survives degraded, not a design point.

- Why the quorum must stay a bare majority, exact, committee 100: with the classical BFT 2/3 quorum (67), $P(\text{stall})$ at 20% sleepy is 0.073% (a stall every ~11 hours) and at 35% sleepy 62% (dead). Your $(Q-1) \times 2$ formula, i.e. majority quorum, is not just compatible with sleepy tolerance, it is essentially forced by it. The price is that the Byzantine margin has to come from somewhere other than the quorum fraction, which is section 4.

4. Malicious nodes: capture, the table that actually binds the committee size

Your goal names "attack in case malicious nodes," so here is the same hypergeometric applied to an adversary. A coalition holding a fraction m of the stake gets a hypergeometric number of committee seats each height. Two thresholds matter, and with a majority quorum they are nearly the same number:

- **Veto** (liveness): with $n - Q + 1$ seats (50 of 100) the coalition can refuse to sign and stall the height. Numerically this is the sleepy tables read at $s = m$, and the outcome is bounded: a half-hour outage into the escape valve, unpleasant, self-healing.
- **Capture** (safety): with Q seats (51 of 100) the coalition alone certifies. Malicious members equivocate freely, so a captured height lets them certify two conflicting blocks, which is a permanent double-certification split (the wedge we know operationally), and it is the primitive behind the cross-chain double-spend from the 4b memo. Honest members cannot counter-certify: with the cap in force, the remaining honest seats are fewer than Q by construction. Anchoring does not arbitrate it, both blocks anchor to the same Bitcoin block. Checkpoints bound how much history it can rewrite, nothing smaller stops it at that height.

The adversary does not need luck on a chosen height; committee draws are public in Option A, so it simply waits for a height it captures and acts then. The per-height probability therefore converts directly into an expected time to the first attack opportunity.

Table 5. $P(\text{coalition captures} \geq \text{quorum})$ per height, population 50,000. Columns are the coalition's share of stake. Two conservatisms: smaller populations make capture strictly harder (the min-stake floor of 40,000 tSEQ actually caps the registry at 10,000 entries, so 50,000 is the worst case beyond worst case), and splitting a given stake across more keys does not raise the coalition's expected seats.

committee	20%	25%	30%	33%	40%	45%
20	0.056%	0.39%	1.71%	3.76%	12.7%	24.9%
30	0.0052%	0.082%	0.64%	1.88%	9.70%	23.1%
50	5e-7	0.0038%	0.093%	0.49%	5.73%	19.7%
80	5e-10	4e-7	0.0056%	0.070%	2.70%	15.6%

committee	20%	25%	30%	33%	40%	45%
100	5e-12	2e-8	0.00088%	0.020%	1.67%	13.4%
150	5e-17	1e-11	9e-8	0.00086%	0.52%	9.45%
200	6e-22	8e-15	1e-9	4e-7	0.17%	6.77%
250	7e-27	5e-18	1e-11	2e-8	0.054%	4.91%
500	1e-51	4e-34	2e-21	4e-15	0.00022%	1.07%

As expected time to the first captured height (30-second blocks):

committee	m=25%	m=30%	m=33.3%	m=40%	m=45%
100	46 yr	39 days	1.8 days	30 min	minutes
150	80,000 yr	10 yr	41 days	1.6 h	minutes
200	never	950 yr	2.4 yr	5 h	minutes
250	never	90,000 yr	53 yr	16 h	minutes
500	never	never	never	156 days	48 min

And as a tolerance statement, the largest coalition a size withstands:

committee	capture rarer than once a decade	rarer than once a century
100	26.0%	24.3%
128	28.5%	27.0%
150	30.0%	28.5%
200	32.3%	31.0%
250	34.0%	33.0%
500	38.5%	37.8%

Readings, and they are the crux of the sizing question:

- **Capture, not sleepiness, is what binds the committee size.** Committee 100 is fine against sleepy nodes at 35%, but a coalition with one third of the stake captures a quorum within two days. Committee 100 defends, for practical horizons, against coalitions up to about a quarter of the stake.
- **The classical one-third Byzantine bound costs about 250 members.** Committee 250 holds

a one-third coalition to one opportunity in ~53 years and a 30% coalition to one in ~90,000 years. Committee 200 gets to 32%, just shy.

- **Above roughly 40% no practical committee defends by sampling.** The margin to the 50% quorum line is too thin; this is the same mathematics as the 45%-sleepy row. Against a near-half coalition the defenses are economic (a 40% stakeholder attacking the chain is burning its own asset; min stake 40,000 tSEQ each, CSV-locked), forensic (with the durable signing record and certificate gossip, a double-certification carries compact cryptographic proof of who equivocated, the natural basis for slashing later), and structural (checkpoints cap the rewrite depth; anchoring means the attack cannot touch Bitcoin legs of swaps that waited for anchor depth). Those defenses exist at any committee size; the table is about what sampling alone guarantees.
- The escape valve is also attack surface: any stall, sleepy or veto-induced, lets a single member certify after the gap, and a coalition always has members in the committee. What keeps this contained is that sub-quorum blocks never gain immediate finality and lose fork-choice to quorum siblings; the 3A fixes below close the remaining abuse of that valve (minting a rival where a certificate exists). It is one more reason to size the committee so stalls stay rare.

5. Dynamic vs fixed, and the value I suggest

The rule that answers both halves of your question:

committee $K = \min(\text{staker pool}, \text{CAP})$, quorum $Q = \text{PosQuorum}(K)$, CAP a fixed constant.

This is dynamic exactly where dynamism helps and fixed exactly where it must be. While the pool is at or below the CAP, the committee is the whole pool: representativeness is perfect (no sampling), capture requires a true majority of stake, disjoint quorums are impossible, and a network of 40 stakers runs 21-of-40. That is your small launch, safe by construction. Past the CAP, the committee stops growing, because the tables say growth stops buying: representativeness saturates (Table 1, flat along each row), sleepy immunity at the design point saturates (Table 3), and the certificate cost, after Option A, is a bitfield, so the only quantity that keeps improving with size is the tolerated coalition share, with sharply diminishing returns after ~250 (34% at 250 against 38.5% at 500).

A population-proportional cap (say committee = pool/10) is strictly worse than a fixed one: below it wastes the perfect-representation regime, above it spends linearly more signatures and latency for a benefit Table 1 shows flattening out.

Suggested CAP: 250, quorum 126. The reasons, in the order the tables give them:

- It is the smallest size that clears the classical one-third Byzantine bound with real margin (34% once-a-decade, 33% once-a-century tolerance).
- Sleepy liveness at the stress point becomes a non-issue: at 35% sleepy, one half-hour outage per ~1.6 years, against one per six hours at CAP 100.
- Representativeness: a one-third region stays within 27.6% to 39.1% of the committee on 95%

of heights.

- Efficiency no longer opposes it. This is what changed my recommendation from the obvious "keep 100": under Option A the certificate is ~300 bytes regardless of K (a 250-bit bitfield is 32 bytes), the gossip round timing at K=250 is $500+25 \times 250 = 6.75$ s of collection window and $700+35 \times 250 = 9.45$ s rounds (pos_producer.cpp:961), comfortable in a 30-second slot, and the schedule sort is microseconds. The old 32 KB-per-block, 257-bytes-per-member cost that made 100 feel like a ceiling is an artifact of private sortition, and it goes away with it.
- It costs nothing until it binds. With min(pool, CAP) the chain runs whole-pool committees until 250 stakers exist; on the current 100-staker fleet the re-genesis chain behaves identically to today (100-member committee, quorum 51).

If you weigh raw committee size harder than the one-third bound, CAP 200 (quorum 101) keeps a 32% tolerance and one 35%-sleepy outage per six weeks; CAP 100 is the floor I would defend, 26% tolerance and 10% downtime if a 35%-sleepy day ever happens. Below 100 the capture column degrades fast (committee 50 loses to a 30% coalition within a day). My pick is 250; the cap is one constant at re-genesis, and every number you need to move it later is in the scan table at the end of the script.

6. 3A explained again, with the example you asked for

First, one precision on your restatement, because it locates the fix exactly. The co-signers do see the full proposed block: a member validates the complete proposal before signing it (TestBlockValidity, pos_producer.cpp:1023) and signs its hash. What a member never learns, if the partition hits, is whether the certificate completed, because the certificate is not part of what it signed: the member set and aggregate signature live in the block's proof solution, which is deliberately excluded from the signed hash so that shares are non-interactive (pos.h:255-266). So the dangerous object is not the block, which the co-signers already hold, it is the few hundred bytes of completed certificate. That asymmetry is the whole problem, and the whole fix.

The failure, step by step. Committee {A, B, C, D}, quorum 3, last agreed height H.

1. Leader A proposes block X at H+1. A, B, C validate X and flood their signature shares. Each of them now holds a full, validated copy of X, and knows only that it emitted a share.
2. A (any node with a quorum of shares may do this) assembles the certificate into X and broadcasts the finished block.
3. The partition lands exactly here: the finished X reaches only A's side. B, C, D emitted shares but never see the completed certificate. They cannot distinguish "X certified" from "X died with the leader."
4. A's side connects X, sees a quorum certificate, marks H+1 immediately final. Finality floor: X.
5. On the other side, nothing arrives at H+1. After the Bitcoin anchor advances 3 blocks, the escaping-stall valve opens, and B, C, D mint a rival Y at H+1 (sub-quorum, legal under the valve). Then at H+2 they produce Z on top of Y with all three signatures: a full quorum. Nobody

signed two blocks at the same height; Z is at a new height.

6. Heal. A's side is pinned to final X at H+1 and rejects the taller Y-Z chain as forking below finality. B, C, D's side has full-quorum Z at H+2 and rejects X. A certified block has been orphaned by an honest supermajority on one side, a finalized block wedges the other: permanent split, no equivocation anywhere, so the durable signing record (the section-1 fix of the last memo) never triggers, and the hash tiebreak never runs because the sides are pinned, not tied.

Tier 1 of the fix: gossip the certificate as its own object, and gate the valve on it.

The certificate is self-verifying and tiny: block hash, signer bitfield, one BLS aggregate, roughly 300 bytes against a block of up to 200 KB. Flood it as an independent gossip message the moment any node assembles it. Two protocol rules attach:

- A node that receives a valid certificate for height H+1 pins its finality floor to that block, exactly as if it had connected it, and fetches the body if it lacks it. Committee members usually need no fetch at all, they validated the proposal before signing; they attach the certificate and connect.
- The escaping-stall valve is suppressed at any height where the node knows a certificate (this is 4a's "do not treat the height as vacant" hold, keyed on certificates instead of anchor verdicts).

Replay the example: at step 3 the 200 KB block is lost to one side, but the 300-byte certificate crosses on any surviving path, and it is retransmitted by every node that has it, not just the assembler. B, C, D receive it, pin to X, connect it (they already hold X), and step 5 never happens. The realistic failure, a lossy flood or a slow link at the wrong moment, is closed, because the object that must cross shrank by three orders of magnitude and gained every gossip path and every holder as a sender.

The honest residual, and Tier 2. If the partition is total, no path exists for even 300 bytes, and it lasts longer than the ~30-minute anchor gap, the B, C, D side still cannot learn of X. At that point no protocol can give both sides progress and safety; the only choice is which one loses. Tier 2 chooses safety, in 4a's style: a member that share-signed X at H+1 must not sign any block, at H+1 or any later height, whose chain excludes X, until it has both waited out the escape gap and actively queried its reachable peers for a certificate at H+1 and heard nothing. In the example, B, C, D hold instead of building on Y-Z: the worst case degrades from a permanent split to the partitioned minority stalling until heal, which is the CAP trade-off you already chose everywhere else in Sequentia (a stall is recoverable, a split is not). The cost is real but bounded: if X genuinely died with its leader (no certificate exists anywhere), the same hold delays that height by the gap plus one query round before the members may back a rival. If you want a wider safety margin at share-signed heights specifically, the gap there can be lengthened (say 6 anchor blocks instead of 3) without touching normal operation. Neither tier reorders a certified block or weakens finality; both prevent the second certificate from forming. Neither is 4b.

7. 3B, the same treatment, and one mechanism that closes both

The mechanics, with the real constants at committee 100: after the round-0 leader's proposal is seen,

nodes collect proposals for $\text{WINDOW} = 500 + 25 \times 100 = 3.0$ s, then rounds of $\text{ROUND} = 700 + 35 \times 100 = 4.2$ s rotate the leadership; the round index is pure wall clock from the round-0 leader's block timestamp (pos_producer.cpp:961-987). When the round index advances, a node re-signs the new round's leader at the same height ($\text{m_signed_round} < \text{round_index}$, pos_producer.cpp:1008). Its earlier-round shares are not revoked by this; they are already flooded and sit in other nodes' collections, assemblable (pos_producer.cpp:1042-1051).

Worked example. Committee 100, quorum 51. Round-0 leader L1's proposal carries timestamp T. Rounds: round 0 ends at $T + 3.0 + 4.2 = T + 7.2$ s.

1. By $T + 7.0$ s, 51 shares for L1's block X have reached one aggregator, which assembles X and starts broadcasting.
2. X's flood takes 1.5 s. At $T + 7.2$ s the clock rolls every node that has not yet connected X into round 1. Suppose 60 members are in that set: they now sign round-1 leader L2's block X' at the same height. Among them are at least 11 members whose round-0 shares for X are in flight or already collected ($51 + 60 > 100$ forces overlap).
3. X' reaches 51 shares by $T + 9$ s and assembles. Both X and X' now carry valid quorum certificates at the same height. The 11 overlap members signed both, honestly: the round schedule told them to.

That is the second honest double-signing channel: no amnesia, no partition even, just a certificate that completes near a round boundary and propagates slower than one round. It breaks the "only via memory loss" reasoning the same way section 2's variance did.

The fix, and it is the same mechanism as Tier 2. State it once, per height:

Having share-signed block X at height H in round r, do not sign any other block at H (any round), and do not sign at any height above H on a chain excluding X, until X's certifiability has lapsed: the round has advanced, you have queried peers for a certificate on X, heard nothing for a grace period, and, for heights (3A), the escape gap has passed.

For 3B the grace period is small, one extra round (4.2 s) plus a certificate query before backing L2, which is exactly the propagation margin the example lacked; a genuinely failed round 0 (leader crashed before assembling) costs one round of delay, which is what round rotation costs anyway. For 3A the lapse condition is the anchor gap plus the query. One rule, two bugs: it removes the protocol-induced re-vote channel entirely (an honest member can no longer hold live shares on two blocks at one height) and converts the 3A residual from split to stall. It also restores the arithmetic your cap gives us: with the cap enforcing overlap 2 and no honest path to a double signature, a same-height double-certification requires two actually-Byzantine members, which is Table 5's problem, not an honest network's.

8. Testing, per your directive: pool larger than committee, no new box

Agreed, and the empirical record supports you, the last three real bugs were caught by scenario, not

review. All of this fits the existing box (the sandbox is lighter than the live 100-node committee):

1. **Deterministic functional test** (style of `feature_pos_certified_sibling_guard.py`, which reproduced the 96/4 race): a regtest cluster with pool > committee under the current threshold rule, netsplit injected, asserting the disjoint-quorum double-certification fires; then the same test against the capped committee, asserting it cannot. Red today, green with the fix: the regression test for section 1.
2. **Sandbox cluster** on the box: ~150 staker processes, cap 30 (quorum 16), the datanote config where the vulnerable window is open on about a third of heights, with induced partitions, to watch a real split happen and then not happen. The same cluster, with stakers tagged EU/US/Asia synthetically, measures representation against Table 1, and with 20/35/45% of processes stopped measures stall frequency against Tables 2 to 4 and the 30-minute outage cost against section 3.
3. **3A/3B race harness**: the partition timed into the share-to-assembly window (3A) and a delayed-flood race at the round boundary (3B), first to reproduce each split, then to verify certificate gossip and the share-lock close them. 3A and 3B were medium-confidence in the last memo precisely because I could not exercise them on the live committee; this is where they become either demonstrated or retired.

9. What I need from you

1. **Section 2 fix**: confirm Option A, the public fixed-size committee (this is the only implementation of your $(Q-1) \times 2$ cap compatible with a small committee), with private VRF retained for leader election.
2. **CAP value**: my recommendation is 250 (quorum 126); 200 and 100 are the documented fallbacks, the trade is Table 5's tolerance column.
3. **Quorum from actual K** = $\min(\text{pool}, \text{CAP})$, and whether you want the odd-K variant that keeps the overlap at 2 for every size.
4. **3A Tier 1**, certificate gossip plus valve suppression: go/no-go.
5. **The unified share-lock** (3A Tier 2 + 3B): go/no-go, and if go, whether the escape gap at share-signed heights stays 3 anchor blocks or widens.
6. The test plan in section 8.

Items 1 to 3 are the re-genesis committee spec; 4 and 5 are node-local prevention rules that could roll earlier, like 4a did. Nothing here reorders certified blocks or weakens finality; every fix is prevention-class. Nothing is implemented yet; on your confirmation I will build it with the tests of section 8 first.

Numbers: `doc/sequencia/committee-sizing-tables.py`, exact hypergeometric and binomial tails, runs anywhere Python does.